

Universidad de San Carlos de Guatemala

Division de las Ciencias de la Ingenieria

Ingenieria en Sistemas

Organizacion De Lenguajes Y Compiladores 1

Seccion: A

Auxiliar: Carlos Fernando Enrique Lopez Garcia



“Manual Tecnico Proyecto Golite Fase 2”

Estudiante:

Mynor Miguel Monzón Martínez 3195097021306

Guatemala, 27 de junio de

2026

1. Introduccion

El presente documento es el Manual Tecnico del Interprete GoLite, desarrollado como Fase 2 del proyecto del curso Organizacion de Lenguajes y Compiladores 1. El interprete esta implementado en Java, utilizando JFlex para el analisis lexico y CUP para el analisis sintactico, con una interfaz grafica construida en Java Swing.

GoLite es un subconjunto del lenguaje Go disenado para explorar los conceptos fundamentales de compiladores: analisis lexico, analisis sintactico, analisis semantico y recorrido del Arbol de Sintaxis Abstracta (AST). La Fase 2 extiende la Fase 1 incorporando slices unidimensionales y multidimensionales, structs con metodos, funciones con paso de parametros por valor y por referencia, sentencia switch-case, for range, reportes de tabla de simbolos y visualizacion del AST mediante Graphviz.

2. Arquitectura del Sistema

2.1 Estructura de paquetes

El proyecto sigue una arquitectura en capas, organizada bajo Maven con la siguiente estructura de paquetes:

```
com.golite
  lexer/      -> Lexer.java (generado por JFlex desde golite.flex)
  parser/     -> Parser.java + Sym.java (generados por CUP desde golite.cup)
  ast/
    NodoAST.java      -> Clase base abstracta del AST
    expresiones/      -> Nodos de expresiones
    sentencias/       -> Nodos de sentencias
  interpreter/
    Interprete.java    -> Singleton principal de ejecucion
    Entorno.java       -> Manejo de scopes y tabla de simbolos
  errors/
    ErrorSemantic.java -> Modelo de error
  reports/
    Graphviz.java      -> Generador de codigo DOT para el AST
    GeneradorAST.java   -> Recorre el AST y produce el DOT
    GeneradorSimbolos.java -> Recolecta simbolos post-ejecucion
  gui/
    VentanaPrincipal.java -> Interfaz grafica Swing
```

2.2 Flujo de ejecucion

El flujo de procesamiento de un archivo .glt sigue las etapas que se describen a continuacion:

Etapas	Componente	Descripcion
1	JFlex / Lexer	Tokenizacion del codigo fuente. Reconoce lexemas, genera objetos Symbol para CUP y registra cada token con su posicion.
2	CUP / Parser	Analisis sintactico LALR(1). Valida la estructura gramatical y construye el AST como una lista de objetos NodoAST.
3	Interprete	Recorre el AST en profundidad. Ejecuta cada nodo evaluando expresiones y aplicando las sentencias correspondientes.
4	Entorno	Gestiona los ambitos anidados mediante una cadena de HashMap. Resuelve y almacena variables con linea, columna y ambito.
5	Reports	GeneradorSimbolos recolecta variables, funciones y structs. GeneradorAST produce el archivo DOT y llama a Graphviz para generar el PNG.
6	GUI Swing	Presenta el editor, la consola, la tabla de tokens, el reporte de errores, la tabla de simbolos y el reporte AST.

3. Analizador Lexico (JFlex)

3.1 Descripcion general

El analizador lexico esta definido en el archivo golite.flex y es generado automaticamente por JFlex 1.9.1. Implementa la interfaz requerida por CUP mediante las directivas %cup y %cupsym. Cada vez que reconoce un lexema, registra el token en el Interprete singleton para el reporte de tokens y devuelve un objeto Symbol con la posicion exacta de linea y columna.

3.2 Definiciones de expresiones regulares

Macro	Expresion Regular	Descripcion
LineTerminator	\r \n \r\n	Fin de linea
WhiteSpace	{LineTerminator} [\t\f]	Espacios en blanco (ignorados)
Digit	[0-9]	Digito decimal
Letter	[a-zA-Z_]	Letra o guion bajo
Identifier	{Letter}({Letter} {Digit})*	Identificador valido
IntLiteral	{Digit}+	Literal entero
FloatLiteral	{Digit}+"." {Digit}+	Literal flotante de 64 bits
StringLiteral	"([^\n] \\.)*"	Cadena entre comillas dobles
RuneLiteral	'([^\n] \\.)*'	Caracter rune entre comillas simples

3.3 Tokens reconocidos

Palabras reservadas:

Lexema	Token	Descripcion
var	VAR	Declaracion de variable con tipo explicito
func	FUNC	Declaracion de funcion o metodo de struct
main	MAIN	Funcion principal de entrada al programa
if	IF	Inicio de sentencia condicional
else	ELSE	Rama alternativa del if
for	FOR	Bucle iterativo
range	RANGE	Iteracion sobre slice en for range
switch	SWITCH	Sentencia switch
case	CASE	Caso dentro de un switch
default	DEFAULT	Caso por defecto en switch
break	BREAK	Interrumpe el bucle o switch actual
continue	CONTINUE	Pasa a la siguiente iteracion del bucle

Lexema	Token	Descripcion
return	RETURN	Retorno de funcion con o sin valor
struct	STRUCT	Definicion de tipo compuesto
true	TRUE	Literal booleano verdadero
false	FALSE	Literal booleano falso
nil	NIL	Valor nulo de slices y structs

Tipos de datos:

Lexema	Token	Tipo en GoLite
int	TINT	Entero de 32 bits con signo
float64	TFLOAT64	Numero de punto flotante IEEE 754 de 64 bits
string	TSTRING	Cadena de caracteres
bool	TBOOL	Valor booleano (true o false)
rune	TRUNE	Caracter Unicode
[]int	LBRACKET RBRACKET TINT	Slice de enteros
[]float64	LBRACKET RBRACKET TFLOAT64	Slice de flotantes
[]string	LBRACKET RBRACKET TSTRING	Slice de cadenas
[]bool	LBRACKET RBRACKET TBOOL	Slice de booleanos
[][]int	LBRACKET RBRACKET LBRACKET RBRACKET TINT	Slice bidimensional de enteros

Funciones embebidas:

Lexema	Token	Descripcion
fmt.Println	PRINTLN	Impresion en consola con salto de linea automatico
strconv.Atoi	ATOI	Conversion de cadena a entero

Lexema	Token	Descripcion
strconv.ParseFloat	PARSEFLOAT	Conversion de cadena a flotante de 64 bits
reflect.TypeOf	TYPEOF	Obtiene el tipo de una expresion en tiempo de ejecucion
append	APPEND	Agrega un elemento al final de un slice
len	LEN	Retorna la longitud de un slice
slices.Index	SLICES_INDEX	Retorna el indice de la primera coincidencia en un slice
strings.Join	STRINGS_JOIN	Une los elementos de un slice de strings con un separador

Operadores:

Lexema	Token	Descripcion
+	PLUS	Suma o concatenacion de cadenas
-	MINUS	Resta o negacion unaria
*	TIMES	Multiplicacion
/	DIV	Division
%	MOD	Modulo (solo entre enteros)
++	INC	Incremento en uno
--	DEC	Decremento en uno
=	ASSIGN	Asignacion simple
:=	DECL_ASSIGN	Declaracion con inferencia de tipo
+=	PLUS_ASSIGN	Asignacion con suma
-=	MINUS_ASSIGN	Asignacion con resta
*=	TIMES_ASSIGN	Asignacion con multiplicacion
/=	DIV_ASSIGN	Asignacion con division
==	EQ	Igualdad
!=	NEQ	Desigualdad
<	LT	Menor que

Lexema	Token	Descripcion
>	GT	Mayor que
<=	LEQ	Menor o igual que
>=	GEQ	Mayor o igual que
&&	AND	Operador logico AND
	OR	Operador logico OR
!	NOT	Negacion logica

3.4 Manejo de errores lexicos

Cualquier caracter que no coincida con ninguna de las reglas anteriores activa la regla catchall [^], que registra un error lexico en el Interprete con el caracter no reconocido, la linea y la columna exactas. El analisis continua sin interrumpirse gracias al modo panico, lo que permite detectar multiples errores en una sola pasada del codigo fuente.

4. Gramatica del Lenguaje GoLite en Notacion BNF

4.1 Descripcion general

La gramatica esta definida en el archivo golite.cup y es procesada por CUP para generar un analizador sintactico LALR(1). La gramatica es no ambigua gracias al uso de llaves obligatorias en todos los bloques y a la declaracion explicita de precedencia y asociatividad de los operadores.

Se utiliza la siguiente convencion en la notacion BNF:

- Los no terminales se escriben en minusculas con guion bajo (ej. stmt_list).
- Los terminales se escriben en mayusculas (ej. IF, LBRACE).
- El simbolo ::= separa la cabeza de la produccion de su cuerpo.
- El simbolo | separa alternativas de una misma produccion.
- La palabra epsilon representa la produccion vacia (cadena vacia).

4.2 Producciones de la gramatica

Simbolo inicial:

```
<program> ::= <top_list> FUNC MAIN LPAREN RPAREN LBRACE <stmt_list> RBRACE
```

Declaraciones globales (structs, funciones, metodos):

```
<top_list> ::= <top_list> <top_decl>
            | epsilon
```

```

<top_decl> ::= <struct_decl>
            | <func_decl>
            | <struct_func_decl>

```

Definicion de struct:

```

<struct_decl> ::= STRUCT ID LBRACE <struct_attrs> RBRACE

```

```

<struct_attrs> ::= <struct_attrs> <struct_attr>
                | <struct_attr>

```

```

<struct_attr> ::= <tipo> ID SEMICOLON
                | <tipo> ID

```

Definicion de funcion y metodo de struct:

```

<func_decl> ::= FUNC ID LPAREN <param_list> RPAREN <tipo> LBRACE <stmt_list>
RBRACE
            | FUNC ID LPAREN <param_list> RPAREN LBRACE <stmt_list> RBRACE
            | FUNC ID LPAREN RPAREN <tipo> LBRACE <stmt_list> RBRACE
            | FUNC ID LPAREN RPAREN LBRACE <stmt_list> RBRACE

```

```

<struct_func_decl> ::= FUNC LPAREN ID <tipo> RPAREN ID LPAREN <param_list> RPAREN
<tipo> LBRACE <stmt_list> RBRACE
                    | FUNC LPAREN ID <tipo> RPAREN ID LPAREN <param_list> RPAREN
LBRACE <stmt_list> RBRACE

```

Lista de sentencias:

```

<stmt_list> ::= <stmt_list> <stmt>
              | epsilon

```

Sentencia:

```

<stmt> ::= <decl_stmt> <opt_semi>
          | <assign_stmt> <opt_semi>
          | <if_stmt>
          | <for_stmt>
          | <switch_stmt>
          | <print_stmt> <opt_semi>
          | <llamada_stmt> <opt_semi>
          | RETURN <expr> <opt_semi>
          | RETURN <opt_semi>
          | BREAK <opt_semi>
          | CONTINUE <opt_semi>
          | LBRACE <stmt_list> RBRACE

```



```

| ID LBRACKET <expr> RBRACKET ASSIGN <expr> <opt_semi>
| ID LBRACKET <expr> RBRACKET LBRACKET <expr> RBRACKET ASSIGN <expr>
<opt_semi>
| ID DOT ID ASSIGN <expr> <opt_semi>
| ID DOT ID DOT ID ASSIGN <expr> <opt_semi>
| ID ID ASSIGN ID LBRACE <inst_valores> <opt_comma> RBRACE <opt_semi>
| ID ID ASSIGN LBRACE <inst_valores> <opt_comma> RBRACE <opt_semi>

```

Declaracion de variables:

```

<decl_stmt> ::= VAR ID <tipo> ASSIGN <expr>
| VAR ID <tipo>
| ID DECL_ASSIGN <expr>

```

Tipos de datos:

```

<tipo> ::= TINT | TFLOAT64 | TSTRING | TBOOL | TRUNE
| LBRACKET RBRACKET TINT
| LBRACKET RBRACKET TFLOAT64
| LBRACKET RBRACKET TSTRING
| LBRACKET RBRACKET TBOOL
| LBRACKET RBRACKET TRUNE
| LBRACKET RBRACKET LBRACKET RBRACKET TINT
| LBRACKET RBRACKET LBRACKET RBRACKET TFLOAT64
| LBRACKET RBRACKET LBRACKET RBRACKET TSTRING
| ID
| LBRACKET RBRACKET ID

```

Asignacion:

```

<assign_stmt> ::= ID ASSIGN <expr>
| ID PLUS_ASSIGN <expr>
| ID MINUS_ASSIGN <expr>
| ID TIMES_ASSIGN <expr>
| ID DIV_ASSIGN <expr>
| ID INC
| ID DEC

```

Sentencia if-else:

```

<if_stmt> ::= IF <expr> LBRACE <stmt_list> RBRACE
| IF <expr> LBRACE <stmt_list> RBRACE ELSE LBRACE <stmt_list> RBRACE
| IF <expr> LBRACE <stmt_list> RBRACE ELSE <if_stmt>

```

Sentencia for:

```

<for_stmt> ::= FOR <expr> LBRACE <stmt_list> RBRACE

```

```

        | FOR <decl_stmt> SEMICOLON <expr> SEMICOLON <assign_stmt> LBRACE
<stmt_list> RBRACE
        | FOR ID COMMA ID DECL_ASSIGN RANGE <expr> LBRACE <stmt_list> RBRACE
        | FOR UNDERSCORE COMMA ID DECL_ASSIGN RANGE <expr> LBRACE <stmt_list>
RBRACE

```

Sentencia switch:

```

<switch_stmt> ::= SWITCH <expr> LBRACE <case_list> RBRACE
                | SWITCH <expr> LBRACE <case_list> DEFAULT COLON <stmt_list> RBRACE

<case_list> ::= <case_list> <case_clause>
                | epsilon

<case_clause> ::= CASE <expr> COLON <stmt_list>

```

Instanciación de struct:

```

<inst_valores> ::= <inst_valores> COMMA <inst_valor>
                | <inst_valor>
                | epsilon

<inst_valor> ::= ID COLON <expr>

```

Expresiones:

```

<expr> ::= <expr> PLUS <expr>
        | <expr> MINUS <expr>
        | <expr> TIMES <expr>
        | <expr> DIV <expr>
        | <expr> MOD <expr>
        | <expr> EQ <expr>
        | <expr> NEQ <expr>
        | <expr> LT <expr>
        | <expr> GT <expr>
        | <expr> LEQ <expr>
        | <expr> GEQ <expr>
        | <expr> AND <expr>
        | <expr> OR <expr>
        | NOT <expr>
        | MINUS <expr> // negacion unaria [%prec UMINUS]
        | LPAREN <expr> RPAREN
        | INT_LIT | FLOAT_LIT | STRING_LIT | TRUE | FALSE | NIL | RUNE_LIT
        | ATOI LPAREN <expr> RPAREN
        | PARSEFLOAT LPAREN <expr> RPAREN
        | TYPEOF LPAREN <expr> RPAREN
        | APPEND LPAREN <expr> COMMA <expr> RPAREN

```

```

| LEN LPAREN <expr> RPAREN
| SLICES_INDEX LPAREN <expr> COMMA <expr> RPAREN
| STRINGS_JOIN LPAREN <expr> COMMA <expr> RPAREN
| ID LPAREN <expr_list> RPAREN
| ID LPAREN RPAREN
| <literal_slice>
| <primary>

```

```

<primary> ::= ID
           | <primary> LBRACKET <expr> RBRACKET
           | ID DOT ID
           | ID DOT ID DOT ID

```

El non-terminal primary garantiza que el acceso a slice (arr[i]) tenga mayor precedencia que los operadores de comparacion, resolviendo el conflicto de parseo en expresiones como arr[i] > arr[j].

Literales de slice:

```

<literal_slice> ::= LBRACKET RBRACKET TINT LBRACE <expr_list> <opt_comma> RBRACE
                | LBRACKET RBRACKET TFLOAT64 LBRACE <expr_list> <opt_comma> RBRACE
                | LBRACKET RBRACKET TSTRING LBRACE <expr_list> <opt_comma> RBRACE
                | LBRACKET RBRACKET TBOOL LBRACE <expr_list> <opt_comma> RBRACE
                | LBRACKET RBRACKET LBRACKET RBRACKET TINT LBRACE <expr_list>
<opt_comma> RBRACE
                | LBRACKET RBRACKET LBRACKET RBRACKET TFLOAT64 LBRACE <expr_list>
<opt_comma> RBRACE
                | LBRACKET RBRACKET LBRACKET RBRACKET TSTRING LBRACE <expr_list>
<opt_comma> RBRACE
                | LBRACE <expr_list> <opt_comma> RBRACE

```

4.3 Precedencia y asociatividad de operadores

Nivel	Operadores	Asociatividad	Descripcion
1 (menor)	.	Izquierda	Acceso a atributo de struct
2	[]	Izquierda	Acceso a elemento de slice (mayor que comparaciones)
3		Izquierda	OR logico
4	&&	Izquierda	AND logico
5	== !=	Izquierda	Igualdad y desigualdad
6	< > <= >=	Izquierda	Operadores relacionales

Nivel	Operadores	Asociatividad	Descripcion
7	+ - (binario)	Izquierda	Suma y resta
8	* / %	Izquierda	Multiplicacion, division, modulo
9 (mayor)	! - (UMINUS)	Derecha	Negacion logica y unaria

5. Arbol de Sintaxis Abstracta (AST)

5.1 Clase base NodoAST

Todos los nodos del AST heredan de la clase abstracta `NodoAST`, que define el metodo abstracto `interpretar()`. Este diseno implementa un patron de ejecucion implicito: cada nodo sabe como ejecutarse a si mismo consultando el Entorno singleton para resolver variables.

```
// NodoAST.java
public abstract class NodoAST {
    public int linea;
    public int columna;
    public NodoAST(int linea, int columna) { ... }
    public abstract Object interpretar();
}
```

5.2 Nodos de expresion

Clase	Descripcion	Atributos clave
NodoEntero	Literal int	int valor
NodoDecimal	Literal float64	double valor
NodoString	Literal string	String valor
NodoBooleano	Literal bool	boolean valor
NodoRune	Literal rune	char valor
NodoNil	Valor nil (null en Java)	-
NodoIdentificador	Acceso a variable por nombre	String nombre
NodoSuma	Suma aritmetica o concatenacion	NodoAST izquierda, derecha

Clase	Descripcion	Atributos clave
NodoResta	Resta aritmetica	NodoAST izquierda, derecha
NodoMultiplicacion	Multiplicacion	NodoAST izquierda, derecha
NodoDivision	Division con verificacion de cero	NodoAST izquierda, derecha
NodoModulo	Modulo int % int	NodoAST izquierda, derecha
NodoComparacion	Operadores ==, !=, <, >, <=, >=	NodoAST izq; String operador; der
NodoLogico	Operadores &&, , !	NodoAST izq; String operador; der
NodoNegacion	Negacion unaria -expr	NodoAST expresion
NodoAtoi	strconv.Atoi(expr)	NodoAST expresion
NodoParseFloat	strconv.ParseFloat(expr)	NodoAST expresion
NodoTypeOf	reflect.TypeOf(expr)	NodoAST expresion
NodoSlice	Literal de slice []tipo{...}	String tipo; List<NodoAST> elementos
NodoAppend	append(slice, valor)	NodoAST slice, valor
NodoLen	len(slice)	NodoAST expresion
NodoSlicesIndex	slices.Index(slice, val)	NodoAST slice, valor
NodoStringsJoin	strings.Join(slice, sep)	NodoAST slice, separador
NodoAccesoSlice	slice[indice]	NodoAST slice, indice
NodoAccesoAtributo	struct.atributo	NodoAST objeto; String atributo
NodoLlamada	funcion(args)	String nombre; List<NodoAST> argumentos
NodoLlamadaMetodo	struct.metodo(args)	NodoAST objeto; String metodo; List args
NodoInstanciaStruct	Tipo {campo: val, ...}	String tipoStruct; List valores

5.3 Nodos de sentencia

Clase	Descripcion	Atributos clave
NodoPrograma	Raiz del AST. Contiene funciones y sentencias de main	List funciones; List sentencias
NodoDeclaracion	Declaracion de variable (var o :=)	String nombre, tipo; NodoAST valor
NodoAsignacion	Asignacion o compuesta (=, +=, -=, *=, /=)	String nombre, operador; NodoAST valor
NodoAsignacionSlice	arr[i] = expr	String nombre; NodoAST indice, valor
NodoAsignacionSlice2D	arr[i][j] = expr	String nombre; NodoAST indiceFila, indiceCol, valor
NodoAsignacionAtributo	struct.campo = expr	String nombreVar, atributo; NodoAST valor
NodoAsignacionAtributoAnidado	struct.campo1.campo2 = expr	String nombreVar, atributo1, atributo2; NodoAST valor
NodoIf	Condicional if / else if / else	NodoAST condicion; List cuerpoIf, cuerpoElse; NodoIf nodoElseIf
NodoFor	Bucle for (while o clasico)	NodoAST condicion, inicializacion, actualizacion; List cuerpo
NodoForRange	Bucle for range sobre slice	String varIndice, varValor; NodoAST slice; List cuerpo
NodoSwitch	Sentencia switch-case-default	NodoAST expresion; List casos; List porDefecto
NodoCaso	Caso dentro de switch	NodoAST valor; List cuerpo
NodoBloque	Bloque anonimo con	List<NodoAST>

Clase	Descripcion	Atributos clave
	ambito propio	sentencias
NodoPrintln	fmt.Println con lista de argumentos	List<NodoAST> argumentos
NodoReturn	Retorno de funcion con o sin valor	NodoAST expresion
NodoBreak	Interrumpe el bucle mediante excepcion interna	linea, columna
NodoContinue	Pasa a la siguiente iteracion mediante excepcion	linea, columna
NodoFuncion	Declaracion de funcion	String nombre, tipoRetorno; List parametros, cuerpo
NodoFuncionStruct	Metodo de struct	String refNombre, tipoStruct, nombre, tipoRetorno; List cuerpo
NodoStruct	Definicion de struct	String nombre; List atributos

6. Interprete y Gestion de Entornos

6.1 Clase Interprete

El Interprete es un singleton que actua como controlador central. Sus responsabilidades son: invocar al lexer y al parser para generar el AST, llamar a `NodoPrograma.interpretar()` para iniciar la ejecucion, mantener la lista de tokens reconocidos, mantener la lista de errores de todas las fases, y proveer la salida de consola a la GUI. En Fase 2 tambien guarda una referencia al ultimo `NodoPrograma` ejecutado para que el reporte AST pueda accederlo sin reejecutar.

```
// Interprete.java (fragmento)
public class Interprete {
    private static Interprete instancia;
    private NodoPrograma ultimoPrograma;

    public String ejecutar(NodoPrograma programa) {
```

```

        this.ultimoPrograma = programa;
        Entorno.resetear();
        try { programa.interpretar(); }
        catch (RuntimeException e) { /* registrar error */ }
        return consola.toString();
    }
    public NodoPrograma getUltimoPrograma() { return ultimoPrograma; }
}

```

6.2 Clase Entorno

La clase Entorno implementa el manejo de ambitos mediante una cadena de HashMap enlazados por referencia padre-hijo. En Fase 2 cada Variable guarda tambien el nombre del ambito, la linea y la columna de su declaracion, para poder generar la tabla de simbolos correctamente.

```

// Entorno.java (fragmento - clase Variable interna)
public static class Variable {
    public String tipo;
    public Object valor;
    public int linea;
    public int columna;
    public String ambito;
}

```

Metodo	Descripcion
pushBloque()	Crea un nuevo HashMap con referencia al scope actual como padre.
pushBloqueConAmbito(nombre)	Igual que pushBloque pero establece el nombre del ambito (usado por funciones).
popBloque()	Restaura el scope al HashMap padre.
declarar(nombre, tipo, valor, linea, col)	Registra la variable en el scope actual con sus metadatos.
obtener(nombre, linea, col)	Busca la variable subiendo por la cadena de padres.
asignar(nombre, valor, linea, col)	Modifica el valor en el scope donde fue declarada.
registrarFuncion(nombre, nodo)	Registra la funcion globalmente. Lanza error si ya existe.
registrarStruct(nombre, nodo)	Registra el struct globalmente. Lanza error si ya existe.

Metodo	Descripcion
recolectarVariables(destino)	Recolecta todas las variables del scope actual y padres para la tabla de simbolos.

6.3 Manejo de break, continue y return

Las sentencias break, continue y return se implementan mediante excepciones internas de Java. Cuando NodoBreak se ejecuta lanza BreakException, capturada por NodoFor y NodoForRange. NodoContinue lanza ContinueException. NodoReturn lanza ReturnException con el valor de retorno. NodoFuncion.ejecutar() captura ReturnException y retorna el valor al caller. Este patron evita la necesidad de flags booleanos en la logica de ejecucion.

6.4 Paso de parametros por referencia

Los structs se pasan por referencia naturalmente ya que en Java los Map son objetos. Las mutaciones de atributos dentro de una funcion o metodo se reflejan directamente en el caller. Los slices (ArrayList de Java) tambien son objetos y se pasan por referencia: modificaciones directas con slice[i] = val dentro de una funcion afectan al caller. El operador append dentro de una funcion requiere reasignacion en el caller ya que retorna una nueva lista.

6.5 Sistema de tipos y coercion implicita

Operandos	Tipo resultante
int op int	int
int op float64	float64
float64 op int	float64
float64 op float64	float64
string + string	string (concatenacion)
int asignado a var float64	conversion automatica a double
nil en operacion aritmetica	error semantico

7. Reportes del Sistema

7.1 Reporte de Tabla de Tokens

Disponible en el menu Reportes > Tabla de Tokens. Muestra todos los tokens reconocidos durante el analisis lexico con las columnas: No., Lexema, Tipo, Linea y Columna.

7.2 Reporte de Errores

Disponible en el menu Reportes > Reporte de Errores. Muestra todos los errores detectados con las columnas: No., Descripcion, Linea, Columna y Tipo (lexico / sintactico / semantico). Los errores lexicos no detienen la ejecucion; el primer error semantico si la detiene.

7.3 Tabla de Simbolos

Disponible en el menu Reportes > Tabla de Simbolos. Se genera despues de la ejecucion y muestra todos los simbolos declarados:

Columna	Descripcion
No.	Numero correlativo
ID	Nombre del simbolo
Tipo Simbolo	Struct, Funcion o Variable
Tipo Dato	int, float64, string, bool, []int, Persona, void, etc.
Ambito	Global o nombre de la funcion donde fue declarado
Linea	Linea de declaracion en el codigo fuente
Columna	Columna de declaracion en el codigo fuente

La implementacion se encuentra en GeneradorSimbolos.java:

```
// GeneradorSimbolos.java (fragmento)
public static String generar() {
    // 1. Structs globales desde Entorno.getStructs()
    // 2. Funciones globales desde Entorno.getFunciones()
    // 3. Variables del entorno post-ejecucion via recolectarVariables()
    return sb.toString();
}
```

7.4 Reporte AST con Graphviz

Disponible en el menu Reportes > Reporte AST. Requiere haber ejecutado el codigo previamente. El proceso es:

- `GeneradorAST.generar(programa)` recorre el AST completo y produce una cadena en formato DOT de Graphviz.
- La cadena se escribe en el archivo `reportes/<nombre_archivo>.dot`.
- Se ejecuta el comando `dot -Tpng` para generar `reportes/<nombre_archivo>.png`.
- La ventana muestra dos tabs: Imagen AST (el arbol visual) y Codigo DOT (el texto para copiar a herramientas online).
- El boton Copiar DOT copia el texto al portapapeles. El boton Abrir carpeta reportes abre el explorador de archivos.

```
// Graphviz.java
public class Graphviz {
    private int count = 0;
    private final StringBuilder body = new StringBuilder();

    public String addNode(String label) {
        String id = "N" + count++;
        body.append(id).append(" [label=\"" + label + "\"];\n");
        return id;
    }

    public void addEdge(String a, String b) {
        body.append(a + " -- " + b + ";\n");
    }

    public String getDot() {
        return "graph AST {\n  node [shape=none];\n" + body + "}\n";
    }
}
```

8. Interfaz Grafica (Java Swing)

8.1 Componentes de la GUI

Componente	Tipo Swing	Funcion
Editor de codigo	JTextArea JScrollPane	Permite escribir y editar archivos .glt con fuente monoespaciada. Soporta apertura y guardado. Muestra numero de linea actual.
Tabs de archivos	JTabbedPane	Permite tener multiples archivos abiertos simultaneamente.

Componente	Tipo Swing	Funcion
Consola de salida	JTextArea (solo lectura)	Muestra la salida de <code>fmt.Println</code> y mensajes del interprete.
Barra de menu	JMenuBar / JMenu	Acceso a Archivo (nuevo, abrir, guardar), Ejecutar y Reportes.
Reportes	JMenuItem x4	Tabla de Tokens, Reporte de Errores, Tabla de Simbolos, Reporte AST.

8.2 Flujo de uso desde la GUI

El usuario escribe o abre un archivo `.glt` en el editor y presiona Ejecutar. La GUI invoca al Interprete singleton, que realiza el analisis lexico, sintactico, semantico y la ejecucion. La consola muestra la salida generada por `fmt.Println`. El menu Reportes permite acceder a la tabla de tokens, reporte de errores, tabla de simbolos y el AST visual generado por Graphviz. Cada vez que se ejecuta un archivo se actualiza el `NodoPrograma` almacenado en el Interprete para que el reporte AST refleje el ultimo programa ejecutado.

9. Herramientas y Configuracion del Proyecto

Herramienta	Version	Uso en el proyecto
Java (JDK)	21	Lenguaje de implementacion. Logica del interprete, AST, reportes y GUI.
JFlex	1.9.1	Generador del analizador lexico a partir de <code>golite.flex</code> .
CUP	11b-20160615	Generador del analizador sintactico LALR(1) a partir de <code>golite.cup</code> .
Apache Maven	3.8.7	Gestion de dependencias, compilacion y generacion del JAR ejecutable.
Graphviz	2.43.0	Herramienta externa para renderizar el AST como imagen PNG desde el archivo DOT.
Java Swing	Incluido en JDK	Framework de interfaz grafica de usuario.
IntelliJ IDEA	Community Ed.	Entorno de desarrollo utilizado.
Ubuntu Linux	24.04	Sistema operativo de desarrollo.

9.1 Construccion del proyecto

El proyecto se compila con Maven desde el directorio GoLiteInterpreter/:

```
cd GoLiteInterpreter/  
mvn clean package
```

Maven ejecuta los plugins de JFlex y CUP durante la fase generate-sources, generando Lexer.java, Parser.java y Sym.java antes de compilar el resto del código. El resultado final es un JAR ejecutable con todas las dependencias incluidas en:

```
target/GoLiteInterpreter-1.0-SNAPSHOT.jar
```

9.2 Ejecucion

```
java -jar target/GoLiteInterpreter-1.0-SNAPSHOT.jar
```

Conclusiones

- El interprete GoLite Fase 2 extiende correctamente la Fase 1 incorporando slices 1D y multidimensionales, structs con metodos, funciones con paso por referencia, switch-case, for range, y los operadores de asignacion compuesta *= y /=.
- La introduccion del non-terminal primary en la gramatica resolvió el conflicto de precedencia del acceso a slice frente a los operadores de comparacion, permitiendo expresiones como `arr[i] > arr[j]` sin ambigüedad.
- Los reportes de Tabla de Simbolos y AST visual con Graphviz cumplen los requerimientos de documentacion del proyecto, generando archivos .dot y .png en la carpeta reportes/ de forma automatica al momento de la ejecucion.
- La clase Entorno fue extendida para registrar linea, columna y ambito en cada Variable, lo que habilita la tabla de simbolos detallada sin costo adicional en tiempo de ejecucion.
- El manejo de duplicados en funciones y structs fue implementado con mensajes de error explicitos, mientras que el primer simbolo registrado permanece activo para permitir la continuacion parcial del programa.
- La arquitectura en capas facilita futuras extensiones del lenguaje GoLite sin modificar los componentes existentes del interprete.